

Revision for Midsem

Lecture 13

February 18, 2026

MPI_Comm_split Example Code Recap

Output
for P=8

```
int newrank, newsize, color = myrank%2;
MPI_Comm newcomm;
MPI_Comm_split (MPI_COMM_WORLD, color, myrank,
&newcomm);
MPI_Comm_rank (newcomm, &newrank);
MPI_Comm_size (newcomm, &newsized);
printf ("%d: %d of %d\n", myrank, newrank,
newsized);
MPI_Comm_free (&newcomm);
```

0: 0 of 4

1: 0 of 4

2: 1 of 4

3: 1 of 4

4: 2 of 4

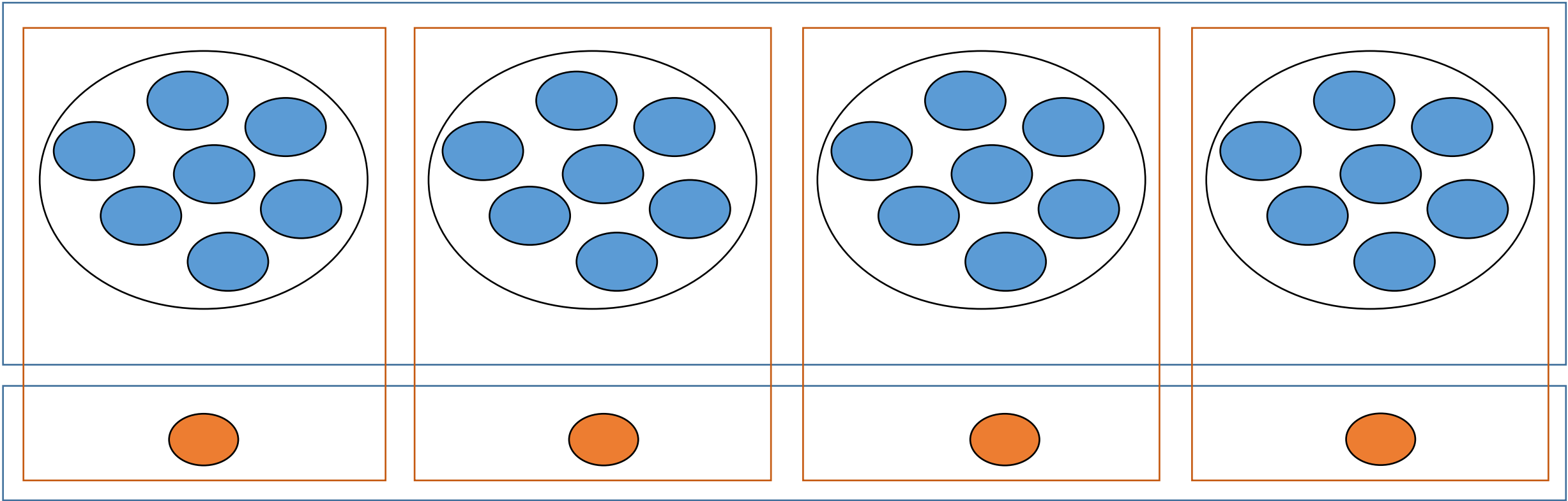
5: 2 of 4

6: 3 of 4

7: 3 of 4

Producer-Consumer Example Recap

An example configuration: 28 simulation processes and 4 analysis processes.
Two splits required: (1) horizontal split and (2) vertical split



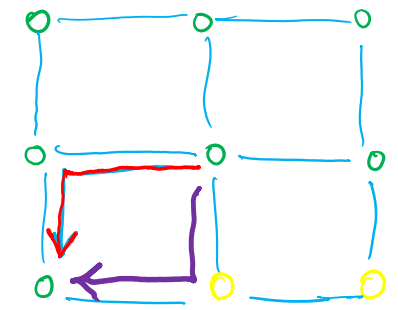
2 communication contexts: 1 for intra-blue or intra-orange process communications, another for the blue->orange producer to consumer process communications.

Domain Decomposition

Process Placement

0	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27

2D virtual process topology for the simulation



For a 5-point stencil, if we request for 7 nodes with $\text{ppn}=4$, what are the number of hops for rank 18 for a 3 X 3 mesh topology (nodes 1 to 7 are used)?

Rank 18 (placed on node 5) communicates with ranks 11, 17 and 19 and 25. These are placed on node numbers 3, 5 and 5 and 7.

Hops 18->11 = 2

Hops 18->17 = 0 (intra-node)

Hops 18->19 = 0 (intra-node)

Hops 18->25 = 2 (the intermediate hop depends on XY vs. YX routing in case of static routing. For example, the path is node 5 -> node 4 -> node 7 in case of XY routing)

Domain Decomposition

Process Placement

0	1	2	3	4	5	6
8	9	10	11	12	13	14
16	17	18	19	20	21	22
24	25	26	27	28	29	30

~~rows X cols 2D virtual process topology~~
rows X cols-1 2D virtual process topology
for the simulation
2D Simulation domain is split among the
blue processes only

Blue -> Simulation processes
Orange -> Analysis processes

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

Process placement on a 2D mesh
topology (rows X cols). In this
example, ppn=1. Note that the
placement will be different if ppn>1

Revision

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

rows X cols processes

- (1) Split into blue and orange groups
- (2) Split row-wise, i.e. form “rows”
number of subgroups

```
int color1 =  
myrank%cols<cols-1?1:0;  
MPI_Comm_split  
(MPI_COMM_WORLD, color1,  
myrank, &newcomm1);
```

```
int color2 = myrank/cols;  
MPI_Comm_split  
(MPI_COMM_WORLD, color2,  
myrank, &newcomm2);
```

Revision

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

Process placement of blue and orange processes

- (1) Split into blue and orange groups
- (2) Split row-wise, i.e. form “rows”
number of subgroups

```
int color1 = ?  
MPI_Comm_split  
(MPI_COMM_WORLD, color1,  
myrank, &newcomm1);
```

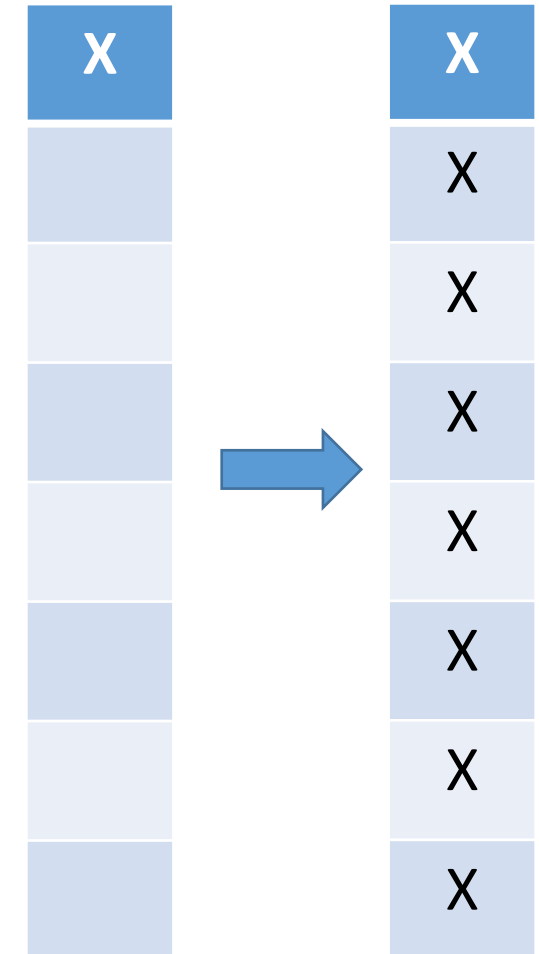
```
int color2 = ?;  
MPI_Comm_split  
(MPI_COMM_WORLD, color2,  
myrank, &newcomm2);
```

Blocking Collectives Revision

- MPI_Barrier
- MPI_Bcast
- MPI_Gather
- MPI_Scatter
- MPI_Reduce
- MPI_Allgather
- MPI_Allreduce
- MPI_Alltoall

Broadcast

- Root process sends message to all processes
- int `MPI_Bcast` (buffer, count, datatype, root, comm)
- “count” is the number of elements in “buffer” – must match
 - “message sizes do not match across processes in the collective routine: Received 400 but expected 4000”
- “buffer” is input at root, output at non-root
- Any process can be a root process but has to be the same process when `MPI_Bcast` is called

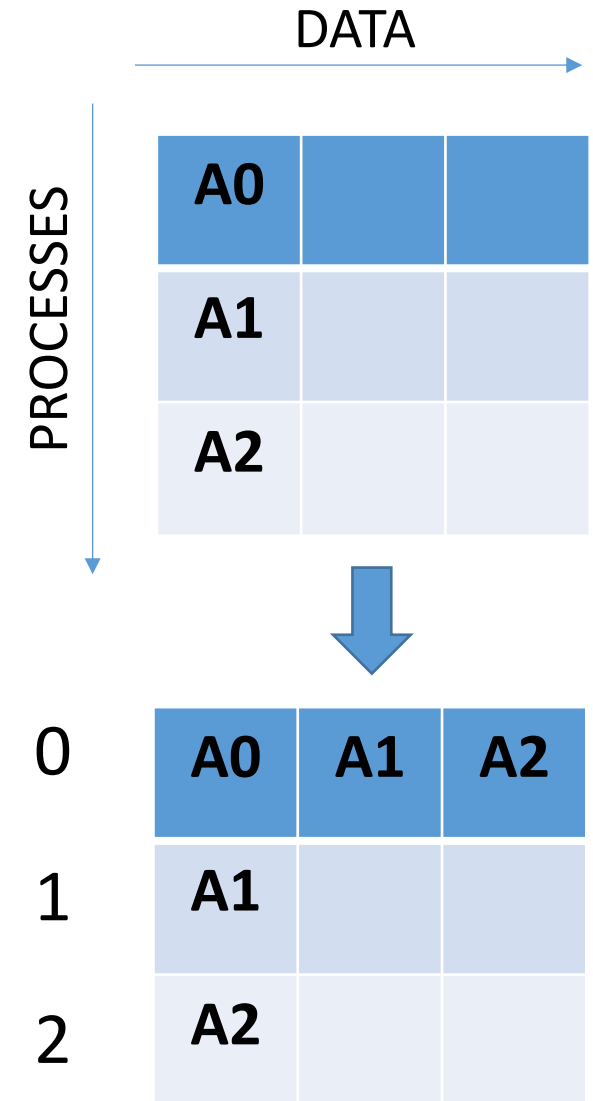


Gather

- Gathers values from all processes to a root process
- int `MPI_Gather` (sendbuf, sendcount, sendtype, *recvbuf*, recvcount, recvtype, `root`, comm)
- Arguments `recv*` not relevant on non-root processes
- `recvcount` → size of any (single) receive
- Distinct values (may be same) received from non-root processes at the root process
- Example: Reading multiple files (1 file per process)

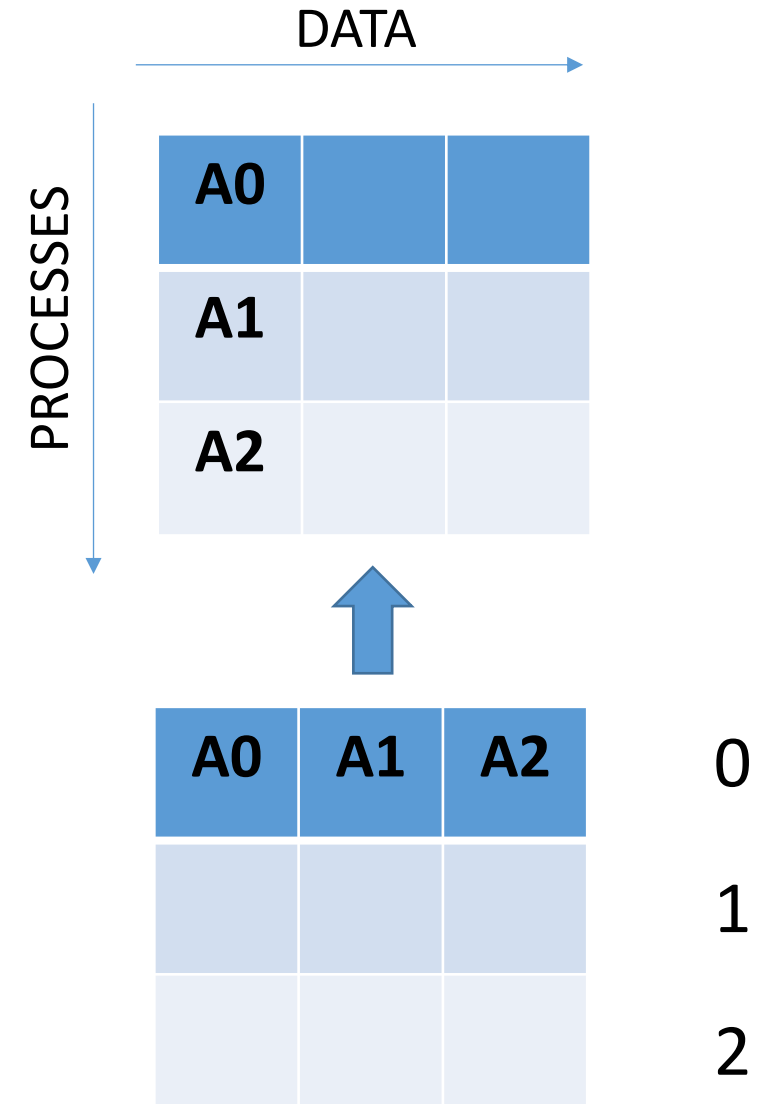
Q: Equivalent point-to-point communications for the same?

- `MPI_Recv` at root
- `MPI_Send` at non-root



Scatter

- Scatters values to all processes from a root process
- `int MPI_Scatter (sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm)`
- Arguments `send*` irrelevant in non-root processes
- `sendcount` – Number of elements sent to each process
- Output parameter – `recvbuf`



Reduce

`MPI_Reduce` (*inbuf*, *outbuf*, count, datatype, op, root, comm)

- Combines element in *inbuf* of each process
- Combined value in *outbuf* of root
- op: MIN, MAX, SUM, PROD, ...
- Example:

`MPI_Reduce (... , 1, MPI_INT, MPI_MAX, ...)` Output: 21

`MPI_Reduce (... , 1, MPI_INT, MPI_MIN, ...)` Output: 1

21

5

1

8

3

2

13

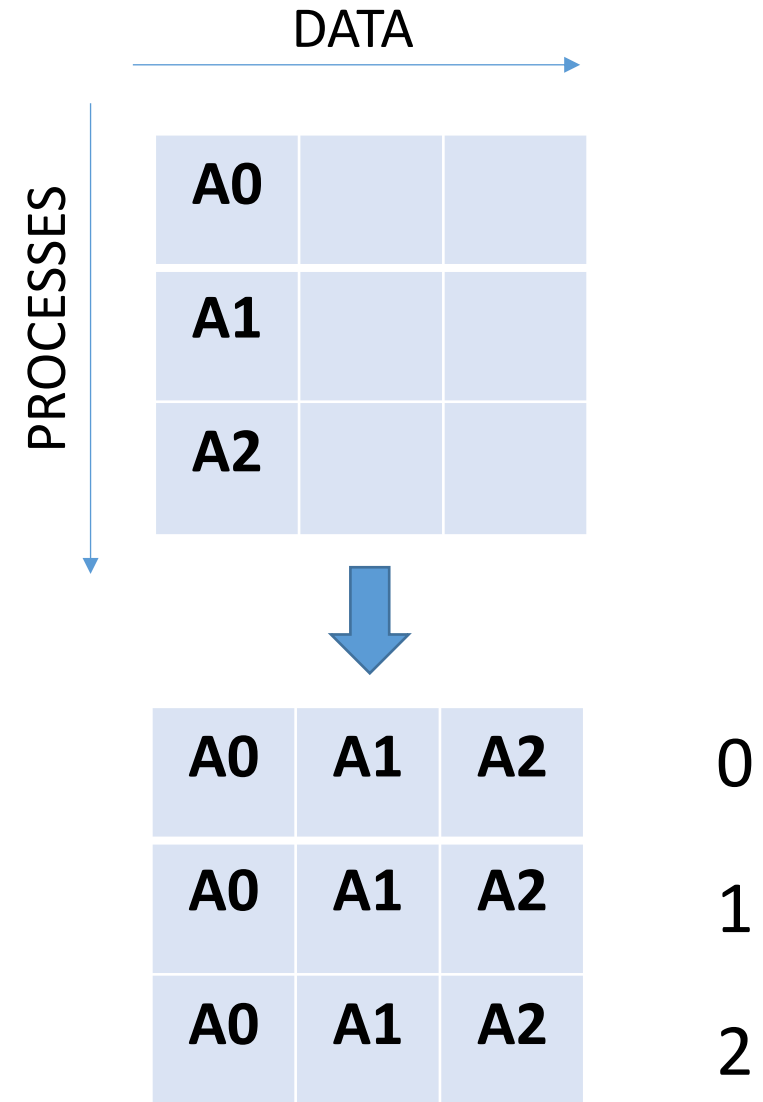
Allgather

- All processes gather values from all processes
- `int MPI_Allgather (sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm)`
- No root process

```
recvbuf = (int *) malloc  
(size*100*sizeof(int));  
MPI_Allgather (sendbuf, 100, MPI_INT,  
recvbuf, 100, MPI_INT, comm);
```

Equivalent collective?

- `MPI_Gather` followed by `MPI_Bcast`



Allreduce

- `MPI_Allreduce` (*inbuf*, *outbuf*, count, datatype, op, comm)
- op: MIN, MAX, SUM, PROD, ...
- Combines element in *inbuf* of each process
- Combined value in *outbuf* of each process

Equivalent collective?

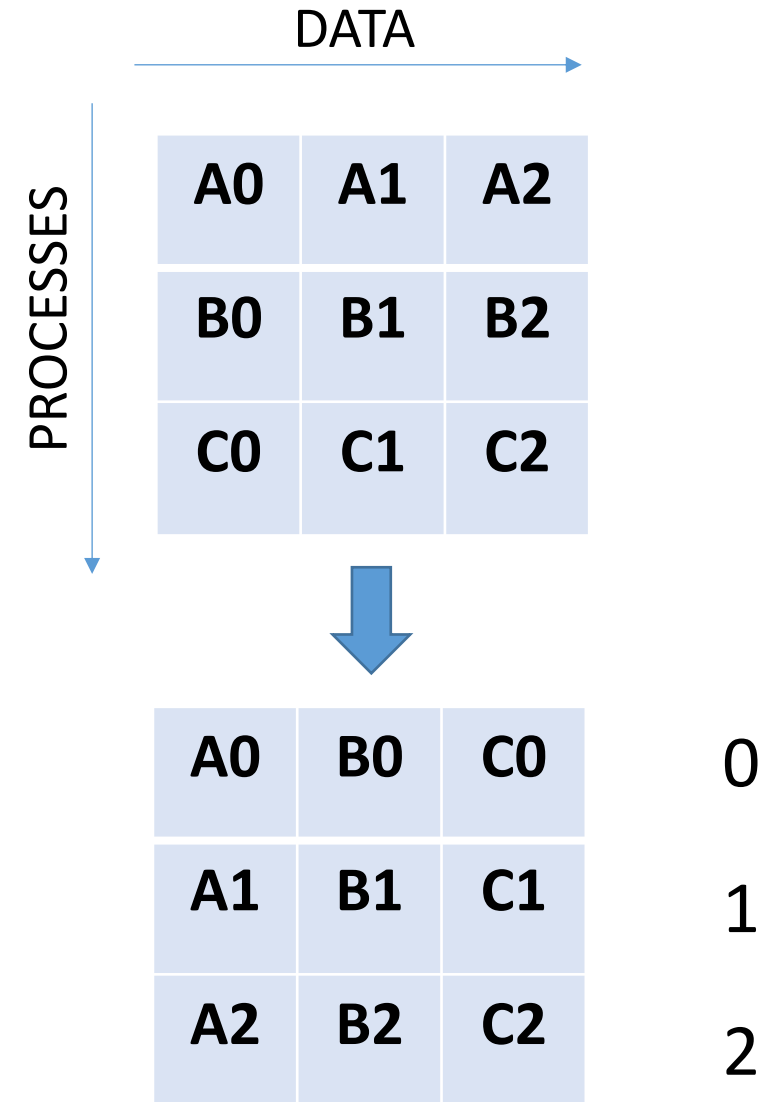
- `MPI_Reduce` followed by `MPI_Bcast`

Alltoall

- Send data from all processes to all processes
- int `MPI_Alltoall` (sendbuf, sendcount, sendtype, *recvbuf*, recvcount, recvtype, comm)
- Output parameter – *recvbuf*
- sendcount/recvcount – sent/received from each process

Equivalent collective

- `MPI_Scatter` from every process



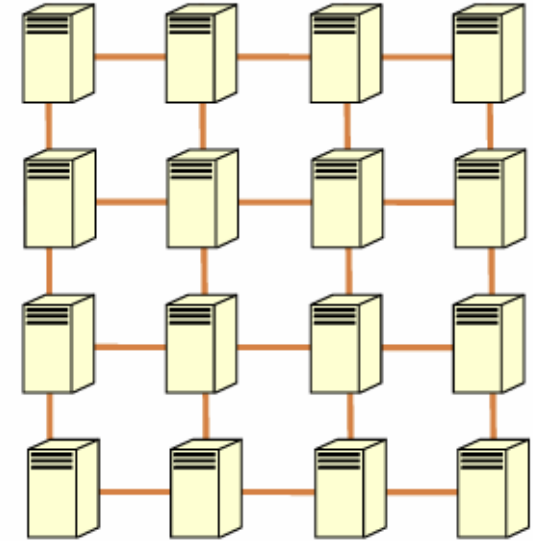
Self-Assessment 2

- Average ~ 6.7

Q1. What is the bisection width of a 2D mesh ($P \times P$)?

Score: 1 Neg. Score: 0

- ✓ ☒ P
- ☐ $P/2$
- ☐ P^2
- ☐ $P-1$



Q2. Source A sends 1024 bytes to Source B in 0.1 s. The effective bandwidth of this communication is:

Score: 1 Neg. Score: 0

- ☐ 1 Kbps
- ✓ ☒ 10 KBps
- ☐ 1 MBps
- ☐ 10 Kbps

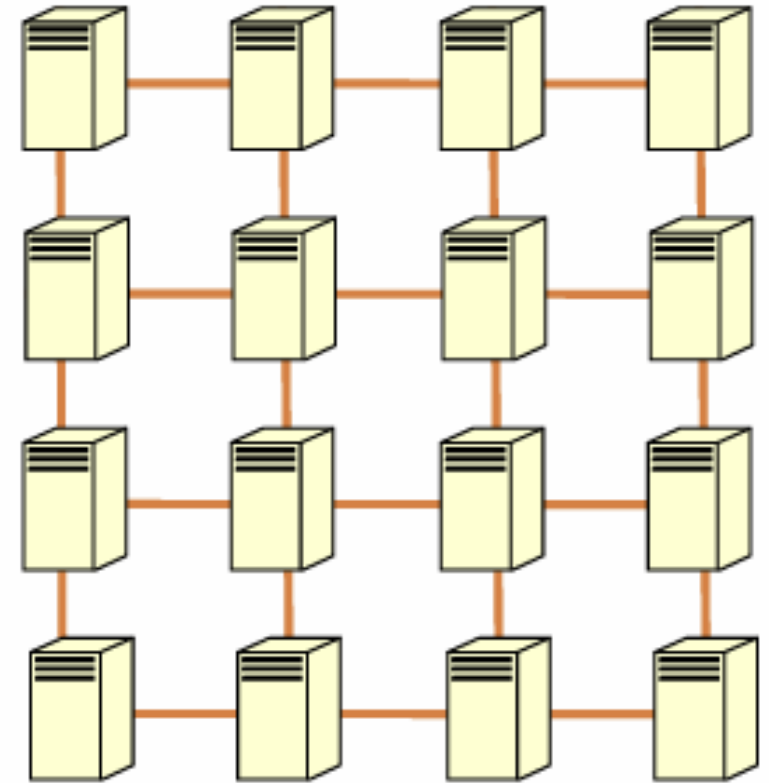
$$\begin{aligned} & 1024 \text{ B} / 0.1 \text{ s} \\ &= 1024 \text{ Bps} \\ &= 1 \text{ KBps} \end{aligned}$$

Q3. Consider a 2D mesh ($P \times P$), with sequential process placement of P^2 processes. The link bandwidth of every link in the mesh is b bytes/sec. The **maximum number of hops for any communication** (say D bytes) on this mesh and the corresponding communication time (assuming latency L per hop and cut-through routing) for that are

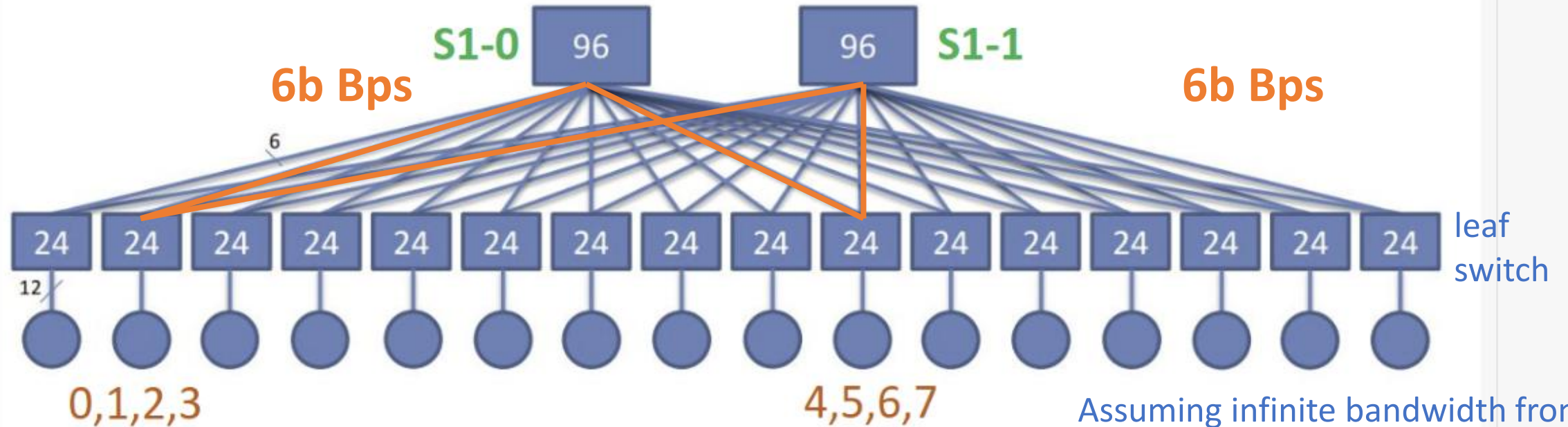
Score: 2 Neg. Score: 0

- ($P-1$) and $L*(P-1) + L*D/b$
- ✓ $2*(P-1)$ and $L*2*(P-1) + D/b$
- $2*(P-1)$ and $L*2*(P-1) + L*D/b$
- ($P-1$) and $L*(P-1) + D/b$

- Maximum #hops for any communication will occur on the diameter [= $2*(P-1)$]
- (P nodes on each side $\rightarrow$ $P-1$ links on each side)
- Communication time as per Hockney model (using cut-through routing) = Latency * number of hops + Data size/Bandwidth



Q4.



There are 12 compute nodes per leaf switch

Assuming infinite bandwidth from
compute node to leaf switch

The maximum available bandwidth for communication between rank 0 and rank 4 (assuming a single link bandwidth of b Bps) is

Score: 1 Neg. Score: 0

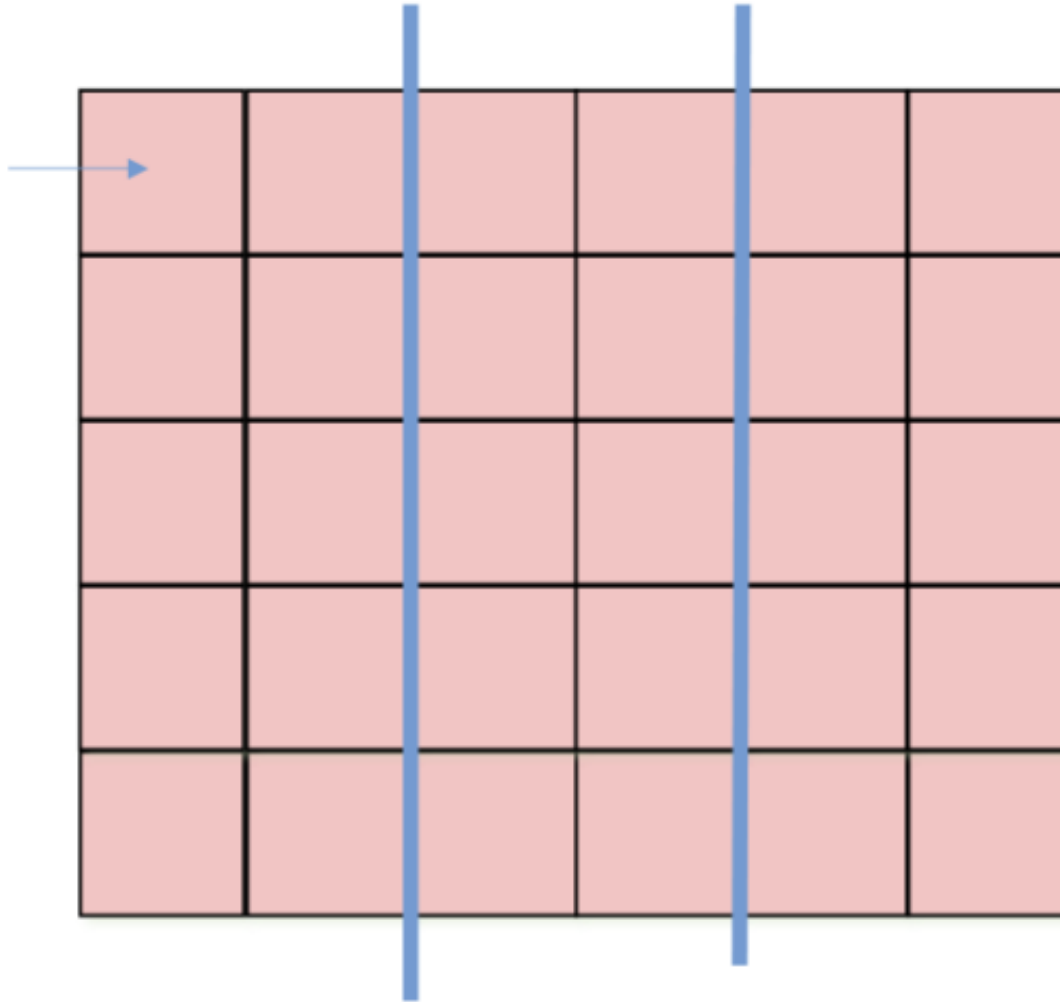
- 6 Bps
- 6b Bps
- 12 Bps
- ✓ 12b Bps

Q5.

2-way 1D NN communication pattern



Grid
point



$$N * N / P$$

✓ $2 * P / N$

$$2 * P / \sqrt{N}$$

$$2 * N$$

#Communications?

$2N$ (N for L + N for R)

#Computations?

$$N * N / P$$

The communication-to-computation ratio of 2-way 1D nearest neighbour communication on $N \times N$ 2D domain using P processes is

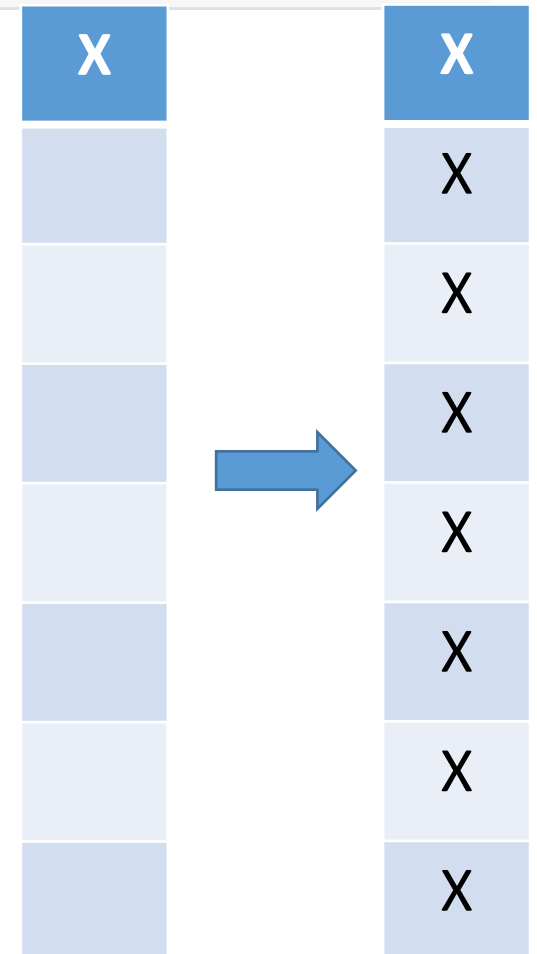
Q6. `MPI_Bcast (data, N, MPI_INT, 0, MPI_COMM_WORLD);`

Select all correct answers about the above function call. Assume N processes.

Score: 2 Neg. Score: 0

- ☐ Rank 0 sends one integer to each of the N processes
- ✓ ☒ Rank 0 sends N integers to each of the N processes
- ☐ Rank 0 receives N integers from each of the N processes
- ✓ ☒ Ranks 1 to N-1 receives N integers from rank 0
- ☐ Ranks 1 to N-1 receives 1 integer from rank 0
- ☐ The code snippet has a bug.

```
int MPI_Bcast( void *buffer, int count, MPI_Datatype
datatype, int root, MPI_Comm comm )
```



Q7. // Assume P = total number of processes

Runs only if P is even, otherwise blocks

```
if (myrank % 2 == 0) {  
    MPI_Send (data, size, MPI_DOUBLE, myrank+1, myrank+1, MPI_COMM_WORLD);  
    MPI_Recv (recvbuf, size, MPI_DOUBLE, myrank+1, myrank, MPI_COMM_WORLD, &status);  
}  
else if (myrank % 2 != 0) {  
    MPI_Recv (recvbuf, myArraySize, MPI_DOUBLE, myrank-1, myrank, MPI_COMM_WORLD, &status);  
    MPI_Send (data, myArraySize, MPI_DOUBLE, myrank-1, myrank-1, MPI_COMM_WORLD);  
}
```

Score: 2 Neg. Score: 0

- ✓ Will not run for P=5
For P=4, the senders are 0 and 2
- ✓ For P=4, the senders are 0, 1, 2, 3
Has a bug, will always block.

Has a bug but will block only when P is odd

Q8. `int newrank, newsize, color = myrank%3;
MPI_Comm_split (MPI_COMM_WORLD, color, myrank, &newcomm);
MPI_Comm_rank (newcomm, &newrank);
MPI_Comm_size (newcomm, &newsize);
printf ("%d: %d of %d\n", myrank, newrank, newsize);`

Score: 3 Neg. Score: 0

Assume size of MPI_COMM_WORLD = 14, the total number of sub-groups formed using the above code snippet is 5

Assume size of MPI_COMM_WORLD = 14, the total number of sub-groups formed using the above code snippet is 4

Assume size of MPI_COMM_WORLD = 14, MPI_Comm_split will fail

✓ Assume size of MPI_COMM_WORLD = 15, ranks 0 and 3 belong to the same sub-group

Assume size of MPI_COMM_WORLD = 15, ranks 0, 1, and 2 belong to the same sub-group

P = 14: (0, 3, 6, 9, 12) (1, 4, 7, 10, 13) (2, 5, 8, 11)

color=0

color=1

color=2

P = 15: (0, 3, 6, 9, 12) (1, 4, 7, 10, 13) (2, 5, 8, 11, 14)

Q9. //Assume P is the total number of processes
int newrank, newsize, color = myrank%4;
MPI_Comm_split (MPI_COMM_WORLD, color, myrank, &newcomm);
MPI_Comm_rank (newcomm, &newrank);
MPI_Comm_size (newcomm, &newsize);
printf ("%d: %d of %d\n", myrank, newrank, newsize);

Score: 2 Neg. Score: 0

- ✓ For P=24, global rank 0 will print 0: 0 of 6
For P=24, global rank 0 will print 0: 0 of 4
- ✓ For P=4, global rank 0 will print 0: 0 of 1
For P=4, global rank 0 will print 0: 0 of 4

P = 24:	(0, 4, ..., 20)	(1, 5, ..., 21)	(2, 6, ..., 22)	(3, 7, ..., 23)	newsize=6
	color=0	color=1	color=2	color=3	
P = 4:	(0)	(1)	(2)	(3)	newsize=1

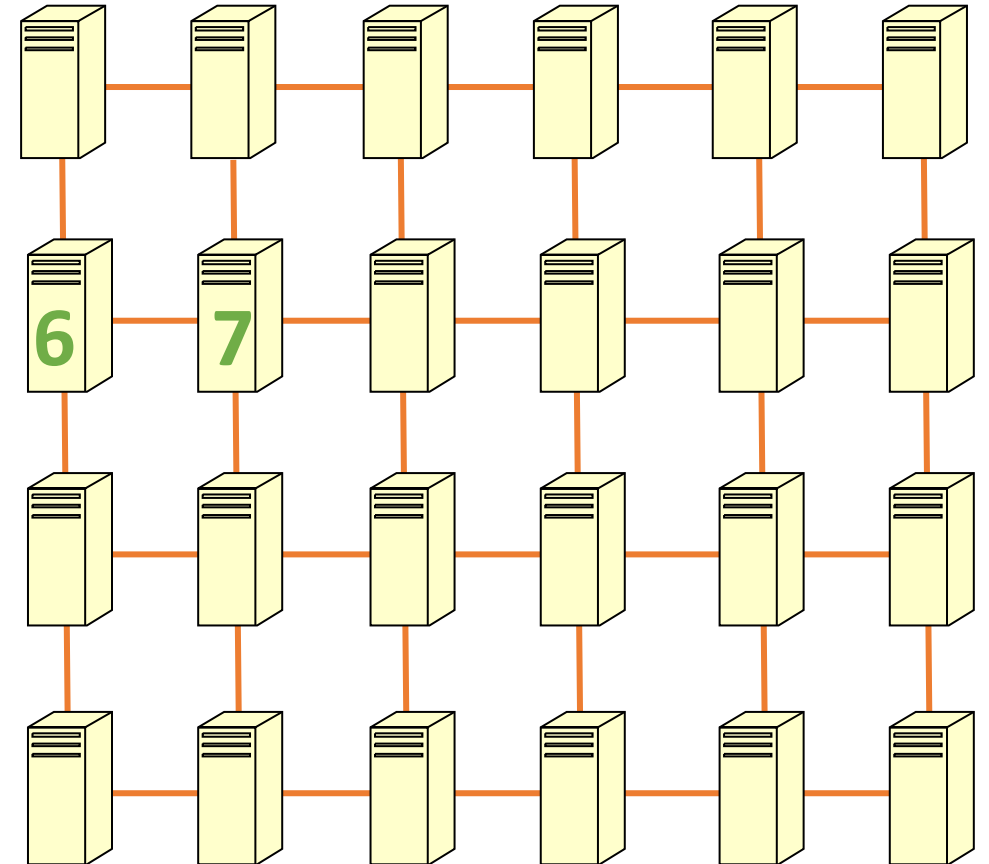
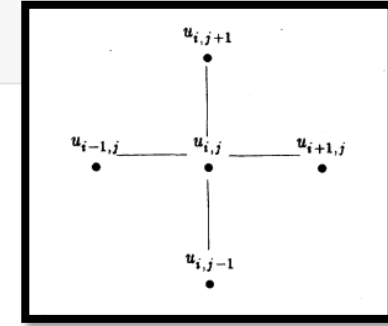
Q10. Consider a 4 X 6 2D node mesh (rows = 4, cols = 6), and sequential placement of 24 processes (ppn = 1) on this mesh.

Score: 2 Neg. Score: 0

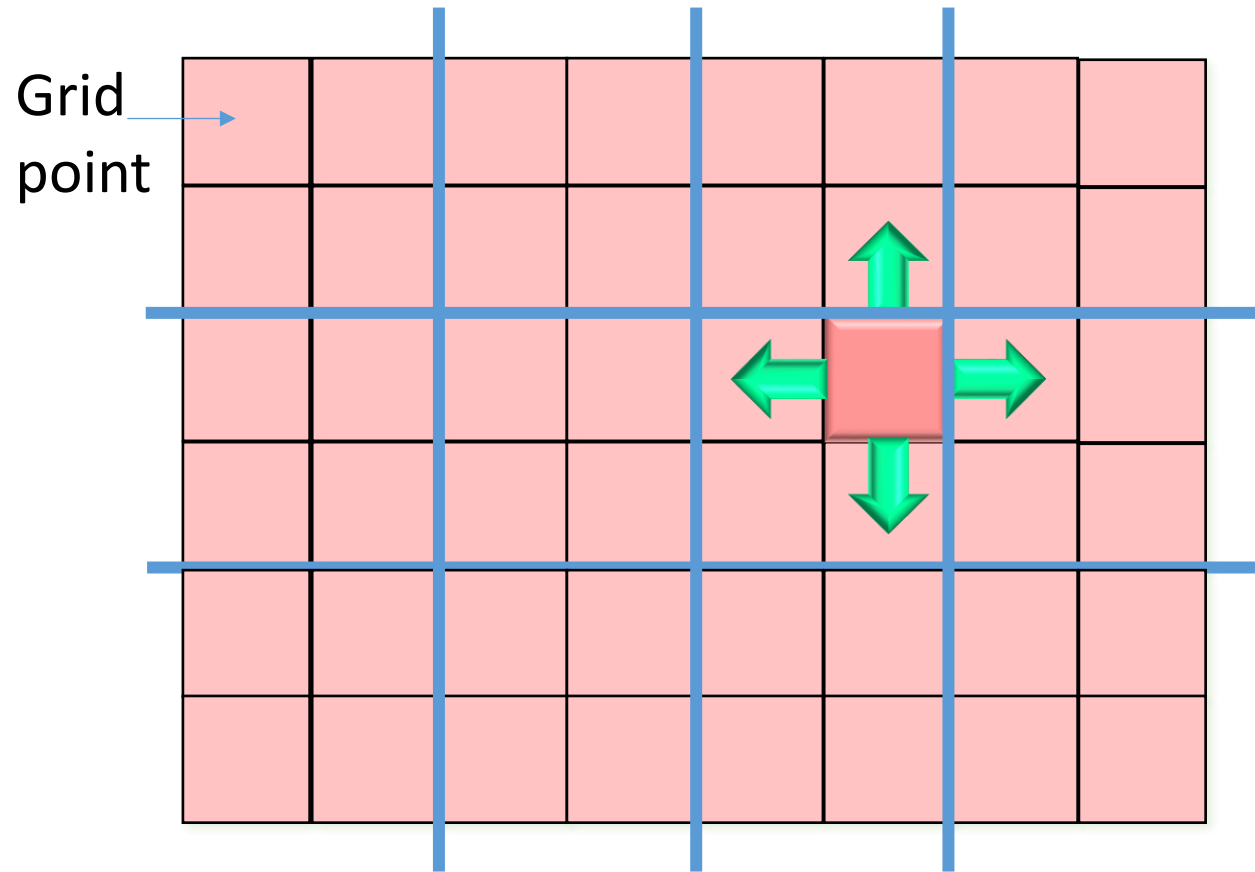
The average number of hops for rank 6 for a 5-point stencil computation is 3

- ✓ The average number of hops for rank 6 for a 5-point stencil computation is 1
- The average number of hops for rank 7 for a 5-point stencil computation is 4
- ✓ The average number of hops for rank 7 for a 5-point stencil computation is 1

- By default, assume XY node numbering
- Rank 6 performs 3 communications (all are 1 hop away), therefore the average is $(1+1+1)/3 = 1$
- Rank 7 performs 4 communications (all are 1 hop away), therefore the average is $(1+1+1+1)/4 = 1$



2D Domain Decomposition of a 2D Domain Recap



4 Sends()
4 Recvs()

N grid points ($\sqrt{N} \times \sqrt{N}$ grid)
 P processes ($\sqrt{P} \times \sqrt{P}$ grid)
 N/P points per process

#Communications?

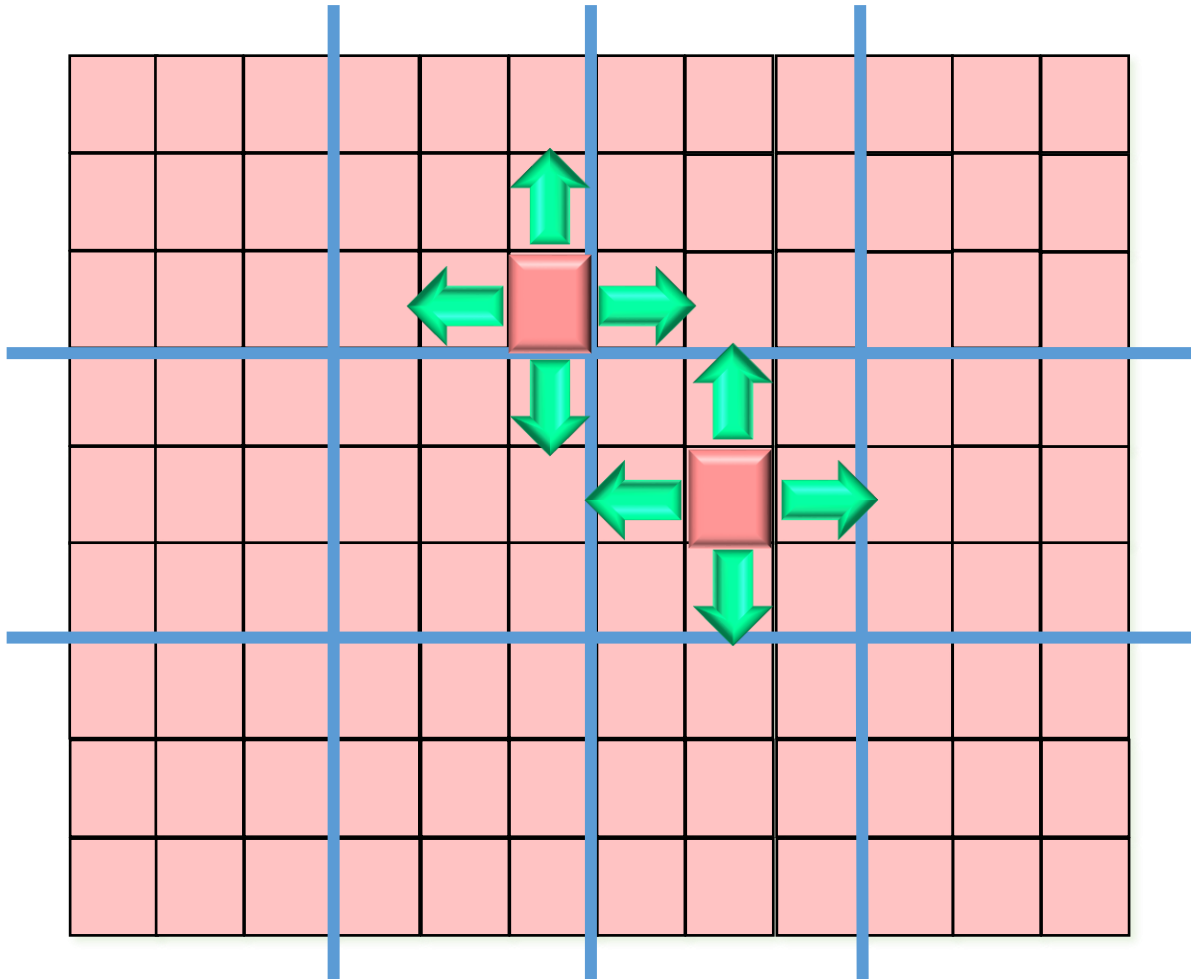
$4\sqrt{N}/\sqrt{P}$ (assuming square grid)

#Computations?

N/P (assuming square grid)

Communication to computation ratio=
 $4\sqrt{P}/\sqrt{N}$

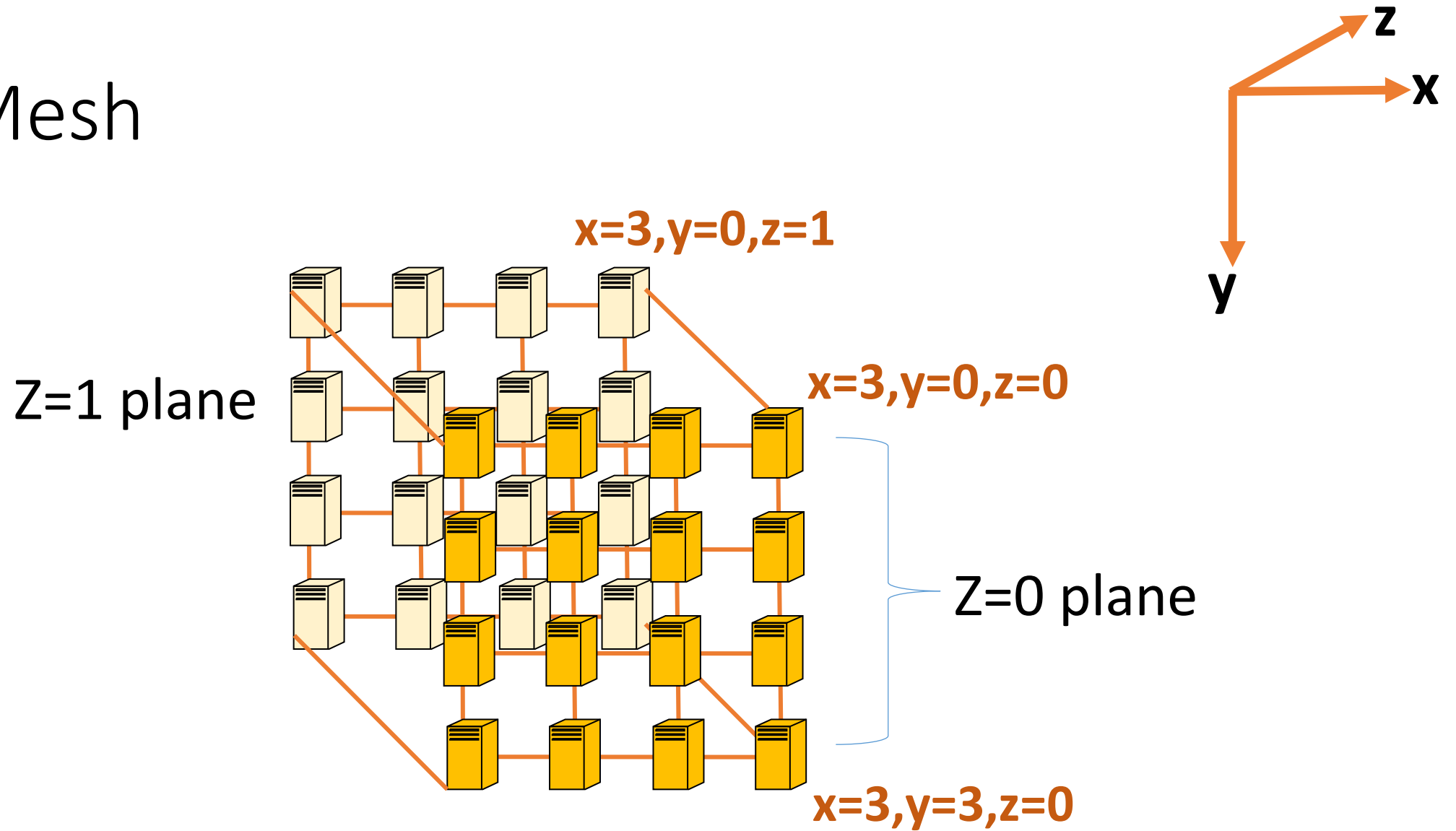
2D Domain Decomposition



M X N grid points
P X Q process grid
 $(M*N)/(P*Q)$ points per process

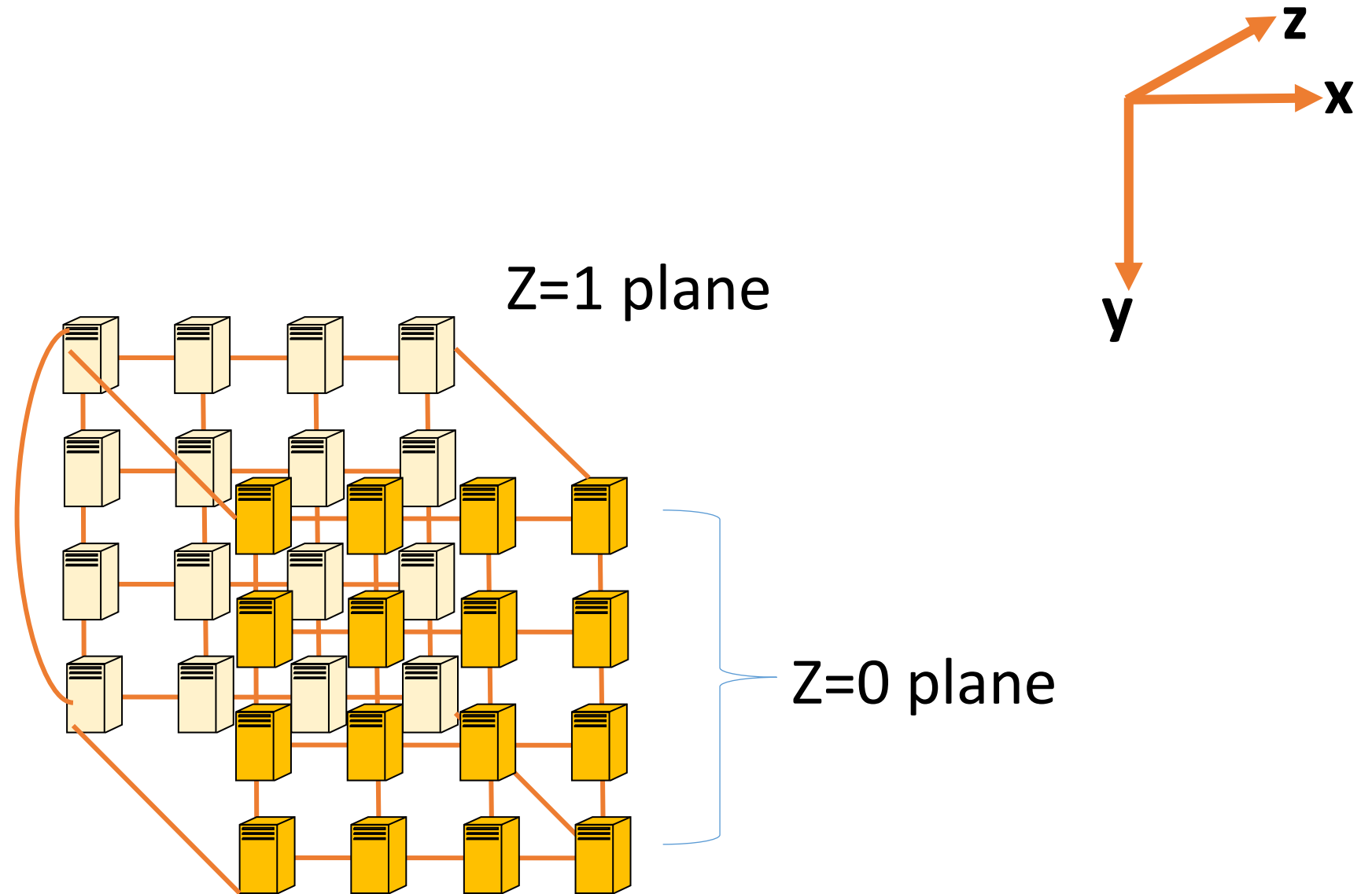
Communication to computation ratio
 $= 2 (M/P + N/Q) / ((M*N)/(P*Q))$

3D Mesh



Note: All network links are not shown to avoid visual clutter, different colours are chosen for nodes/links for visual clarity

3D Torus



Note: All network links are not shown to avoid visual clutter, different colours are chosen for nodes/links for visual clarity

Homework – Derive the network parameters

- 3D Mesh ($\sqrt[3]{p} \times \sqrt[3]{p} \times \sqrt[3]{p}$)
- 3D Torus ($\sqrt[3]{p} \times \sqrt[3]{p} \times \sqrt[3]{p}$)